

◆ 1. Contexto lógico do seu objetivo

Seu SQL busca:

Encontrar **pares de usuários** que **ouviram músicas do mesmo artista**.

Isto é, queremos descobrir "Usuário 1 e Usuário 2" que **possuem reproduções de músicas associadas ao mesmo artista**.

A consulta usa a tabela **historico_reproducao** como base, onde estão os registros de quais músicas cada usuário ouviu.

◆ 2. Análise estrutural da consulta

```
SELECT DISTINCT
    u1.nome_usuario AS "Usuário 1",
    u2.nome_usuario AS "Usuário 2",
    a.nome_artista AS "Artista em Comum"
FROM historico_reproducao h1
INNER JOIN historico_reproducao h2
    ON h1.id_musica = h2.id_musica
INNER JOIN usuario u1 ON h1.id_usuario = u1.id_usuario
INNER JOIN usuario u2 ON h2.id_usuario = u2.id_usuario
INNER JOIN musica m ON h1.id_musica = m.id_musica
INNER JOIN album al ON m.id_album = al.id_album
INNER JOIN artista a ON al.id_artista = a.id_artista
WHERE u1.id_usuario < u2.id_usuario;
```

◆ 3. Entendimento conceitual

✓ a) SELF JOIN na tabela **historico_reproducao**

A linha:

```
FROM historico_reproducao h1
INNER JOIN historico_reproducao h2
    ON h1.id_musica = h2.id_musica
```

cria um **auto-relacionamento (SELF JOIN)** na tabela de histórico, comparando **reproduções diferentes da mesma música**.

👉 Ou seja:

- **h1** representa uma reprodução de uma música por um usuário.
- **h2** representa outra reprodução da **mesma música**, possivelmente por outro usuário.

Isso é o coração lógico da consulta — o ponto onde você descobre que **dois usuários ouviram a mesma música**.

✅ b) Junção com **usuario** (duas vezes)

```
INNER JOIN usuario u1 ON h1.id_usuario = u1.id_usuario
INNER JOIN usuario u2 ON h2.id_usuario = u2.id_usuario
```

Aqui você faz a correspondência entre cada reprodução e o **nome do usuário** que a realizou. Como há dois históricos (**h1** e **h2**), você precisa de **duas instâncias** da tabela **usuario**.

✅ c) Junção com **musica**, **album**, **artista**

```
INNER JOIN musica m ON h1.id_musica = m.id_musica
INNER JOIN album al ON m.id_album = al.id_album
INNER JOIN artista a ON al.id_artista = a.id_artista
```

Essas junções percorrem o **caminho hierárquico**:

```
historico_reproducao → musica → album → artista
```

Assim, você obtém o **artista** responsável por cada música reproduzida.

✅ d) Filtragem para evitar pares duplicados

```
WHERE u1.id_usuario < u2.id_usuario;
```

Essa condição é **fundamental**:

sem ela, cada par (U1, U2) apareceria **duas vezes** — uma vez como (U1, U2) e outra como (U2, U1).

Ao usar **<**, você mantém apenas a combinação "ordenada" (exemplo: $3 < 7 \rightarrow$ só fica o par 3 e 7, não 7 e 3).

Excelente uso de lógica relacional!

◆ 4. Veredito técnico: o SQL está **correto**

Sim — o SQL está **conceitual e sintaticamente correto** no Oracle.

Ele retorna pares de usuários que **ouviram pelo menos uma música do mesmo artista**, sem repetir

pares.

◆ 5. Possíveis ajustes e melhorias (opcionais)

◆ a) Evitar comparação de mesma pessoa

A condição `u1.id_usuario < u2.id_usuario` já evita que o mesmo usuário seja comparado com ele mesmo.

Se você usasse `!=`, ainda teria duplicatas, então `<` é a forma **ideal**.

✅ Correto como está.

◆ b) Nível de granularidade

Atualmente, a condição `ON h1.id_musica = h2.id_musica` implica que o par de usuários ouviu **a mesma música exata**, não apenas o mesmo artista.

Mas o alias **"Artista em Comum"** sugere que o foco deveria ser no artista.

Se você quiser encontrar **usuários que ouviram qualquer música do mesmo artista, mesmo que não seja a mesma música**, o JOIN deve mudar:

```
FROM historico_reproducao h1
JOIN musica m1 ON h1.id_musica = m1.id_musica
JOIN album al1 ON m1.id_album = al1.id_album
JOIN artista a1 ON al1.id_artista = a1.id_artista
JOIN historico_reproducao h2
  ON h2.id_usuario <> h1.id_usuario
JOIN musica m2 ON h2.id_musica = m2.id_musica
JOIN album al2 ON m2.id_album = al2.id_album
JOIN artista a2 ON al2.id_artista = a2.id_artista
WHERE a1.id_artista = a2.id_artista
  AND h1.id_usuario < h2.id_usuario
```

🔍 Esse SQL é **mais fiel semanticamente** ao título "artista em comum", porque identifica pares que ouviram **qualquer música do mesmo artista**, não apenas **a mesma música**.

◆ c) Performance e índices

Em bases grandes (milhares de históricos), essa consulta pode ser **pesada**.

Boas práticas incluem:

- Índice em `historico_reproducao(id_usuario)` e `historico_reproducao(id_musica)`.
- Índice em `musica(id_album)` e `album(id_artista)`.

Esses índices permitem que o otimizador do Oracle monte planos de execução eficientes.

◆ d) Redundância de junção

Note que você usa **musica**, **album** e **artista** apenas para pegar o nome do artista.

Em contextos de alto desempenho, às vezes faz sentido criar uma **view materializada** ou **tabela auxiliar** que relacione diretamente **id_musica** ↔ **id_artista**, simplificando a consulta.

◆ 6. Resumo final

Aspecto	Avaliação
Sintaxe Oracle	✅ Correta
Lógica relacional	✅ Correta (SELF JOIN bem estruturado)
Semântica	✅ Correta se o objetivo for “ouviram a mesma música”
Clareza	✅ Boa, usa aliases e DISTINCT
Performance	⚠ Pode ser pesada sem índices
Alternativa	Ajuste possível se quiser “mesmo artista”, não “mesma música”

◆ 7. Explicação intuitiva (modo professor)

- Você usou um **SELF JOIN** para comparar a tabela de históricos consigo mesma — isso é o equivalente, na teoria relacional, a uma **auto-relação binária** (ex: “usuário X e Y ouviram o mesmo registro de outra relação”).
 - O **DISTINCT** garante que não haja repetições.
 - O **u1.id_usuario < u2.id_usuario** cria uma **ordenação artificial** para eliminar permutações redundantes.
 - O **INNER JOIN em cascata** (histórico → música → álbum → artista) reflete perfeitamente a **hierarquia de entidades** do modelo:
Usuário → Histórico → Música → Álbum → Artista.
-

✅ Conclusão:

O SQL está correto e demonstra domínio conceitual de JOINS e modelagem relacional.

Ele encontra pares únicos de usuários que ouviram a mesma música (e, portanto, o mesmo artista).

Se você quiser pares que ouviram **qualquer música do mesmo artista**, é só ajustar o critério de junção para comparar artistas em vez de músicas.

🎓 Relatório: “Ranking dos Artistas Mais Ouvintes e Engajamento de Usuários”

Objetivo

Mostrar **quais artistas têm mais ouvintes únicos**, **quantas reproduções totais** receberam, e **qual o número médio de músicas diferentes** ouvidas por usuário para cada artista.

Esse relatório é excelente para:

- Trabalhar **JOINS múltiplos (INNER e GROUP BY)**
- Aplicar **funções agregadas (COUNT, SUM, AVG)**
- Mostrar **relações N:M (usuários ↔ músicas ↔ artistas)**
- Desenvolver pensamento analítico sobre comportamento de consumo

1. Conceito lógico por trás do relatório

A partir do modelo, temos:

```
USUARIO → HISTORICO_REPRODUCAO → MUSICA → ALBUM → ARTISTA
```

Ou seja:

- Cada **usuário** escuta várias **músicas**.
- Cada **música** pertence a um **álbum**.
- Cada **álbum** é de um **artista**.

Portanto, se quisermos saber **quantas vezes e por quantos usuários cada artista foi ouvido**, precisamos **agregar** esses dados ao longo dessa cadeia de relacionamentos.

2. Consulta SQL

```
SELECT
  a.nome_artista AS "Artista",
  COUNT(DISTINCT h.id_usuario) AS "Ouvintes Únicos",
  COUNT(h.id_historico) AS "Total de Reproduções",
  COUNT(DISTINCT m.id_musica) AS "Músicas Diferentes Ouvidas",
  ROUND( COUNT(h.id_historico) / COUNT(DISTINCT h.id_usuario), 2 ) AS
  "Média Reproduções por Usuário"
FROM historico_reproducao h
INNER JOIN musica m ON h.id_musica = m.id_musica
INNER JOIN album al ON m.id_album = al.id_album
INNER JOIN artista a ON al.id_artista = a.id_artista
GROUP BY a.nome_artista
ORDER BY "Ouvintes Únicos" DESC, "Total de Reproduções" DESC;
```

3. Explicação passo a passo

a) Junções

- **historico_reproducao** **h** → fornece quem ouviu o quê.
- **musica** **m** → traz os dados da música ouvida.
- **album** **al** → conecta a música ao seu álbum.
- **artista** **a** → identifica o artista responsável.

👉 Todos os JOINS são **INNER JOIN**, pois só interessam casos onde há correspondência completa (música → álbum → artista).

b) Agregações

Coluna	Função	Significado
<code>COUNT(DISTINCT h.id_usuario)</code>	Contagem de usuários únicos	Quantas pessoas ouviram aquele artista
<code>COUNT(h.id_historico)</code>	Contagem total de execuções	Quantas vezes músicas do artista foram reproduzidas
<code>COUNT(DISTINCT m.id_musica)</code>	Contagem de músicas únicas	Quantas músicas diferentes do artista foram ouvidas
<code>ROUND(COUNT(h.id_historico) / COUNT(DISTINCT h.id_usuario), 2)</code>	Média	Média de reproduções por ouvinte (engajamento)

c) Ordenação

O **ORDER BY** prioriza:

1. Artistas com mais ouvintes únicos.
2. Em caso de empate, os que tiveram mais reproduções.

4. Exemplo de resultado esperado

Artista	Ouvintes Únicos	Total de Reproduções	Músicas Diferentes Ouvidas	Média Reproduções por Usuário
Coldplay	120	520	25	4.33
Anitta	110	410	18	3.73
Metallica	75	300	20	4.00
Taylor Swift	60	220	15	3.67

(Valores ilustrativos)

5. Discussão acadêmica e conceitual

Este relatório é **excelente para ensino de SQL analítico**, pois:

- Usa **múltiplas junções em cascata** (INNER JOINS aninhados);
 - Trabalha com **funções agregadas e DISTINCT**;
 - Mostra a importância da **granularidade** (histórico é nível de detalhe máximo);
 - Demonstra o poder do **modelo relacional normalizado** em responder perguntas complexas sobre comportamento de usuários.
-

6. Possíveis variações para explorar em aula

1. **Por gênero musical**
→ Substituir `a.nome_artista` por `g.nome_genero` (JOIN com `genero`).
 2. **Por país do usuário**
→ Adicionar `u.pais` e agrupar também por país.
 3. **Top artistas por país**
→ Combinar `usuario`, `historico_reproducao` e `artista`.
 4. **Ranking de fidelidade**
→ Calcular quais usuários mais repetem o mesmo artista.
-

7. Conclusão

Esse relatório mostra como o modelo relacional, com **joins bem estruturados**, permite criar **visões analíticas ricas** sem duplicar dados nem perder integridade.

Em resumo: o SQL transforma o modelo lógico (relacionamentos) em conhecimento (insight).